Hazelnut quality control: telling good nuts from defective ones

Mykhailo Shelestiuk · 13 November 2026

Goal

I wanted to train a neural network that looks at a photo of a hazelnut and says whether it is good (OK) or defective (NG), and to understand each tool I used along the way. I also wanted it to run on my own GPU, so everything is set up around that, and all files are kept on drive E: because drive C: is almost full.

Setup

Windows 11, Intel i9-13800, NVIDIA RTX 4060 Laptop (8 GB). Python 3.14.10 in a virtual environment, with PyTorch 2.11.0 and torchvision 0.26.0 (CUDA 12.8 build), Pandas 3.1.1, scikit-learn 1.9.1 and Pillow 13.0. PyTorch does the training on the GPU, Pandas holds the table of photos, and scikit-learn splits the data. The first install pulled a CPU-only PyTorch from PyPI by mistake. I replaced it with the CUDA build and confirmed that a computation really runs on the RTX 4060.

The project was built with help from an AI coding assistant (Devin).

Data

The photos come from the hazelnut category of the MVTec AD dataset (Bergmann et al., CVPR 2019): 501 images of 1024×1024 pixels. 431 are good. 70 are defective: 18 cracks, 17 cuts, 18 holes and 17 with a print on the shell. So only 14% of the photos show a defect. The license is CC BY-NC-SA 4.0, which means non-commercial use only.

The dataset ships with a train folder that contains no defective nuts, because it was made for a different kind of task. Since I train on both classes, I ignored that split and made my own. The script *step2_pandas.py* reads all folders, labels each photo 0 (good) or 1 (defect), and splits the photos 70/15/15 into train, validation and test, keeping the share of each defect type equal in all three parts. The random seed is fixed (42), so the split is the same every time. It also checks that every file exists, that no path repeats and that no two files have identical content.

Part	Good	Defective	Total	Used for
train	302	48	350	learning
validation	64	11	75	choosing the best epoch
test	65	11	76	final check, used once

The same script works for all 15 categories of MVTec (5,354 photos), so I built tables for all of them.

Two things went wrong with data. First, I had a folder of 30 sample photos (*sim_camera*). Comparing the files byte by byte showed they are copies of MVTec hazelnut photos, and 18 of them were in my training part, so they say nothing about how well the model generalizes. Second, a pre-generated set of 528 augmented images turned out to be inconsistent (file names collided, and copies of one photo could land in both training and test), so I dropped those files and apply augmentation on the fly, to training photos only.

Model and training

Training a network from zero needs far more photos than I have, so I used transfer learning. ResNet18, already trained on ImageNet, knows how to pick out edges, textures and shapes. I replaced its last layer

with a single output and fine-tuned the whole network on the 350 training photos. Photos are resized to 256×256 and normalized the same way ImageNet was.

During training each photo is randomly flipped, rotated and slightly brightened or darkened, so the network sees a new version every epoch. The loss is binary cross-entropy where a missed defect counts 6.3 times more than a wrong call on a good nut (302 good divided by 48 defective). Without that weight the network could score 86% just by answering OK every time. I used the AdamW optimizer with a learning rate of 0.0001, batches of 32 and 25 epochs. After each epoch the model is tested on the validation photos, and the epoch with the best balanced accuracy is saved (balanced accuracy is the average of the recall on good photos and on defective ones, so it can't be inflated by ignoring the rare class).

The network outputs one raw number. A sigmoid turns it into a probability of defect, and a photo is called NG if that probability is 50% or more. The threshold can be changed in the application without retraining.

Results

On the 76 test photos, which the model never saw during training, it found all 11 defective nuts and flagged none of the 65 good ones. The validation set gave the same result. The saved model's best epoch is 18 of 25.

I ran the same method on the wood category (326 photos, 9 defective in the test set). It found 9 of 9 defects and had no false alarms on 40 good photos. Training took about a second per epoch.

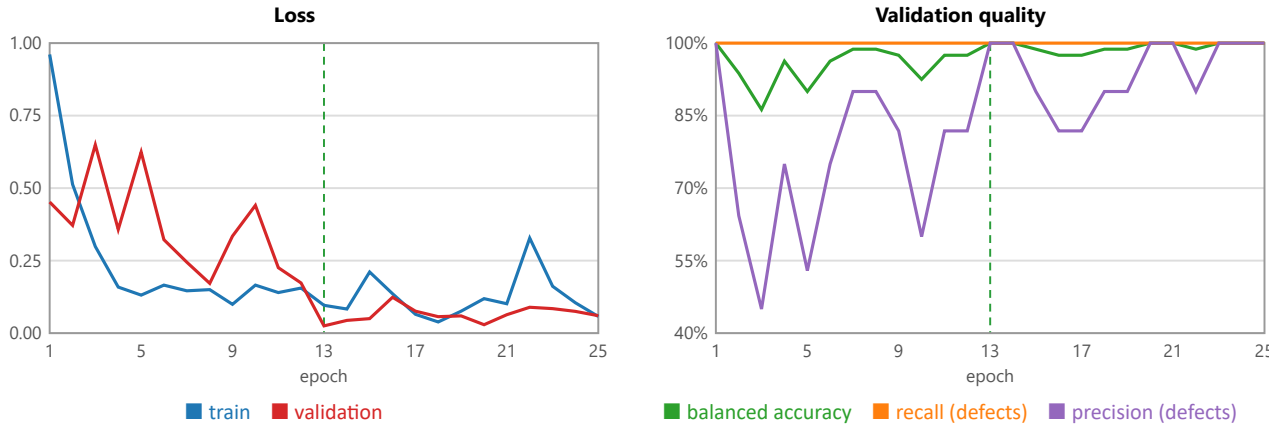


Figure 1. Training the wood model. Precision on the validation set jumps around at first because it only has 9 defective photos, so one wrong call moves it by roughly ten points. The dashed line marks the saved epoch (13).

These numbers look great, but the test sets are tiny. Zero misses out of 11 only tells me the true miss rate is probably below about 27%, and for wood below about 33%. The photos are also lab-quality: same light, same background, same camera. On a real line I would expect worse results until the model is retrained on photos from the actual camera. And because defects are rare in production, even a small false-alarm rate means many of the nuts flagged would actually be good.

The application

To avoid running everything from the command line I wrote a small Windows application (Python and tkinter) that runs the same scripts. It has three tabs. **Data** builds and shows the dataset table with counts per part and a preview of each photo. **Training** starts and stops training with the chosen settings and shows GPU load and live charts of loss and quality. When a run ends it shows the result on the test photos, and the new model replaces the old one only after I confirm. **Check** takes one photo, a folder, or a random test photo, and shows the verdict with a threshold slider and the steps from raw score to probability to answer. A shortcut on the desktop starts it.

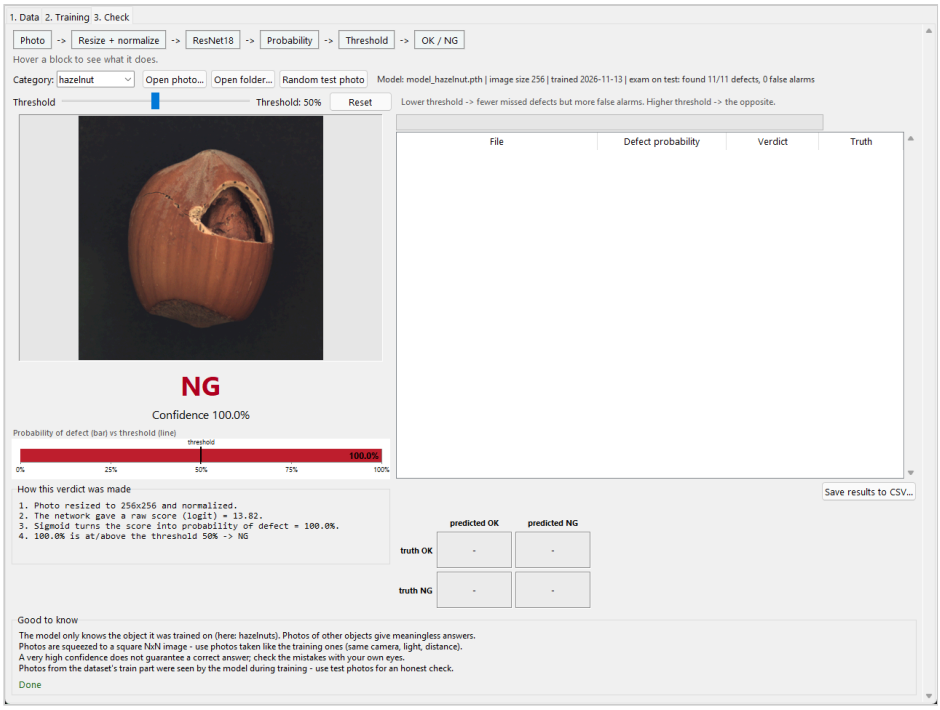


Figure 2. Check tab: a hazelnut with a crack is flagged NG.

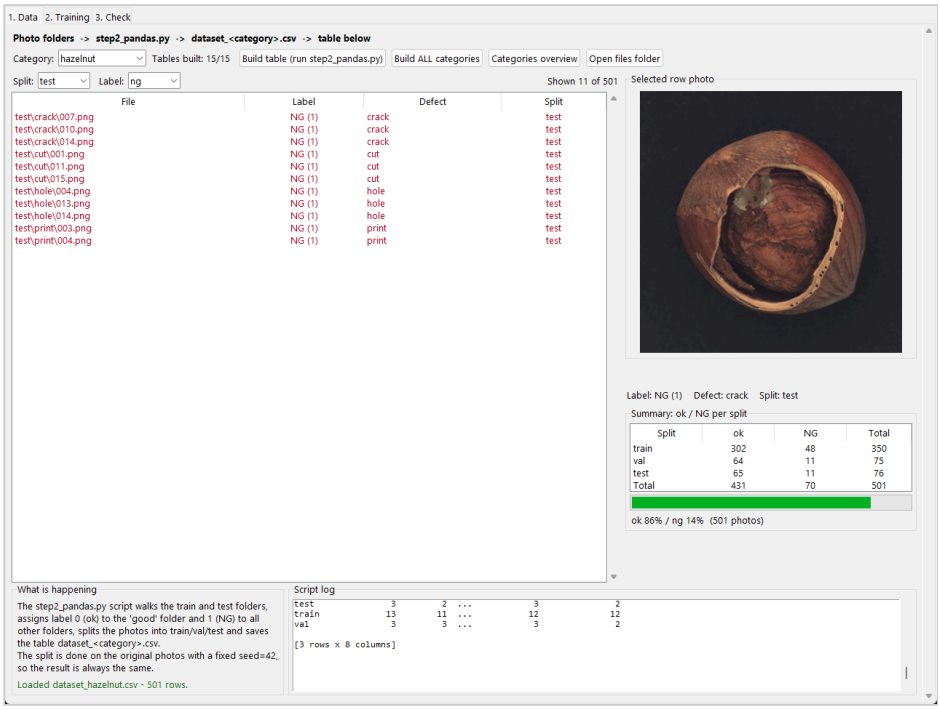


Figure 3. Data tab: the table of photos, with the counts for each part.

Reference

P. Bergmann, M. Fauser, D. Sattlegger, C. Steger. MVTec AD – A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection. CVPR 2019.

K. He, X. Zhang, S. Ren, J. Sun. Deep Residual Learning for Image Recognition. CVPR 2016.